Library Management System

Database Design Documentation

Prepared by: Sanjeev Sharma

Database: MySQL

1. Introduction

The Library Management System (LMS) is a database designed to manage day-to-day library operations such as maintaining book records, tracking members, issuing and returning books, managing fines, and handling book reservations. This document explains the database structure in a simple way, covering all 12 tables, their columns, and how they are connected to each other.

1.1 Objectives

- Maintain records of books, authors, publishers, and categories
- Keep track of library members and staff
- Manage book issue and return process
- Track fines for late returns or damaged books
- Allow members to reserve books that are currently unavailable

2. Database Overview

The database consists of 12 tables in total. These tables are grouped into four simple categories for easy understanding:

Category	Tables Included
Book Information	Books, Authors, Publishers, Categories, BookAuthors, BookCopies
People	Members, Staff
Transactions	IssueRecords, Returns, Fines
Requests	Reservations

3. Table Structure (Schema Details)

Below is the column-level detail for every table, along with the primary key (PK) and foreign key (FK) references.

3.1 Publishers

Stores details of book publishers.

Column Name	Data Type	Notes
publisher_id	INT	Primary Key
publisher_name	VARCHAR(100)	Not Null
address	VARCHAR(200)	
phone	VARCHAR(15)	
email	VARCHAR(100)	

3.2 Categories

Stores book categories or genres.

Column Name	Data Type	Notes
category_id	INT	Primary Key
category_name	VARCHAR(50)	Not Null
description	VARCHAR(200)	

3.3 Authors

Stores author information.

Column Name	Data Type	Notes
author_id	INT	Primary Key
first_name	VARCHAR(50)	Not Null
last_name	VARCHAR(50)	
nationality	VARCHAR(50)	
date_of_birth	DATE	

3.4 Books

Stores details of each book title.

Column Name	Data Type	Notes
book_id	INT	Primary Key
title	VARCHAR(200)	Not Null
isbn	VARCHAR(20)	Unique
publisher_id	INT	FK -> Publishers
category_id	INT	FK -> Categories
publication_year	YEAR	
edition	VARCHAR(20)	
language	VARCHAR(30)	
price	DECIMAL(8,2)	

3.5 BookAuthors

Junction table linking books and authors (many-to-many).

Column Name	Data Type	Notes
book_author_id	INT	Primary Key
book_id	INT	FK -> Books
author_id	INT	FK -> Authors

3.6 BookCopies

Stores individual physical copies of each book.

Column Name	Data Type	Notes
copy_id	INT	Primary Key
book_id	INT	FK -> Books
rack_number	VARCHAR(20)	
status	ENUM	Available/Issued/Reserved/Damaged/Lost
purchase_date	DATE	

3.7 Members

Stores library member details.

Column Name	Data Type	Notes
member_id	INT	Primary Key
first_name	VARCHAR(50)	Not Null
last_name	VARCHAR(50)	
email	VARCHAR(100)	Unique
phone	VARCHAR(15)	
address	VARCHAR(200)	
membership_date	DATE	
membership_type	ENUM	Student/Faculty/General
status	ENUM	Active/Inactive/Suspended

3.8 Staff

Stores library staff/librarian details.

Column Name	Data Type	Notes
staff_id	INT	Primary Key
first_name	VARCHAR(50)	Not Null
last_name	VARCHAR(50)	
email	VARCHAR(100)	Unique
phone	VARCHAR(15)	
role	VARCHAR(50)	
join_date	DATE	

3.9 IssueRecords

Tracks which member borrowed which book copy.

Column Name	Data Type	Notes
issue_id	INT	Primary Key
copy_id	INT	FK -> BookCopies
member_id	INT	FK -> Members
staff_id	INT	FK -> Staff
issue_date	DATE	Not Null
due_date	DATE	Not Null
status	ENUM	Issued/Returned/Overdue

3.10 Returns

Tracks book return details.

Column Name	Data Type	Notes
return_id	INT	Primary Key
issue_id	INT	FK -> IssueRecords
return_date	DATE	Not Null
condition_status	ENUM	Good/Damaged/Lost
remarks	VARCHAR(200)	

3.11 Fines

Tracks penalty amounts for late or damaged returns.

Column Name	Data Type	Notes
fine_id	INT	Primary Key
issue_id	INT	FK -> IssueRecords
member_id	INT	FK -> Members
amount	DECIMAL(8,2)	Not Null
reason	VARCHAR(100)	
paid_status	ENUM	Paid/Unpaid
paid_date	DATE	

3.12 Reservations

Tracks member requests to reserve unavailable books.

Column Name	Data Type	Notes
reservation_id	INT	Primary Key
book_id	INT	FK -> Books
member_id	INT	FK -> Members

reservation_date	DATE	Not Null
status	ENUM	Pending/Fulfilled/Cancelled

4. Table Relationships

The tables are connected using primary key and foreign key relationships as shown below:

Relationship	Type
Books -> BookCopies	One book has many copies (1:N)
Books <-> Authors (via BookAuthors)	Many-to-many (M:N)
Publishers -> Books	One publisher has many books (1:N)
Categories -> Books	One category has many books (1:N)
Members -> IssueRecords	One member has many issue records (1:N)
BookCopies -> IssueRecords	One copy can be issued many times (1:N)
Staff -> IssueRecords	One staff member handles many issues (1:N)
IssueRecords -> Returns	One issue record has one return (1:1)
IssueRecords -> Fines	One issue record may generate one fine (1:1 / 1:N)
Members -> Fines	One member can have many fines (1:N)
Members -> Reservations	One member can make many reservations (1:N)
Books -> Reservations	One book can have many reservations (1:N)

5. Recommended Table Creation Order

Tables should be created in this order so that foreign key references do not fail:

- 1. Publishers
- 2. Categories
- 3. Authors
- 4. Books
- 5. BookAuthors
- 6. BookCopies
- 7. Members
- 8. Staff
- 9. IssueRecords
- 10. Returns
- 11. Fines
- 12. Reservations

6. Simple Workflow

- A member registers and is added to the Members table.
- Books, authors, and publishers are added to their respective tables.
- When a member borrows a book, a record is added in IssueRecords.

- When the book is returned, a record is added in Returns and the status in IssueRecords is updated.
- If the book is returned late or damaged, a record is added in Fines.
- If a book is unavailable, the member can add a request in Reservations.

End of Document